Dynamic Priority Scheduling

Real-Time Operative Systems Course

Paulo Pedreiras, DETI/UA/IT

October 19, 2026



1 Preliminaries

2 Online scheduling with dynamic priorities

3 Analysis: CPU utilization bound

4 Analysis: CPU Load Analysis

5 Other deadline assignment criteria

Last lecture



Fixed-priority online scheduling

- Priority assignment:
  - Rate-Monotonic Scheduling
  - Deadline-Monotonic Scheduling
  - and arbitrary priorities (e.g importance- or criticality-based)
- Schedulability analysis:
  - The CPU utilization bound
  - Worst-Case Response-Time analysis

Agenda for today

- Online scheduling with dynamic priorities
  - Earliest Deadline First scheduling
  - Analysis: CPU utilization bound and CPU Load Analysis
- Optimality and comparison with RM
  - Schedulability level, number of preemptions, jitter and response time
- Other dynamic priority criteria
  - Least Slack Time (LST) / Least Laxity First (LLF)

Note: some authors define laxity as equivalent to slack, while others define laxity as a dynamic parameter - amount of time the task can be delayed from the current time without missing its deadline - and slack as a static parameter - difference between the task's relative deadline (D_i) and its worst-case execution time (C_i). As such, some papers refer interchangeably to LLF and LST.

1 Preliminaries

2 Online scheduling with dynamic priorities

3 Analysis: CPU utilization bound

4 Analysis: CPU Load Analysis

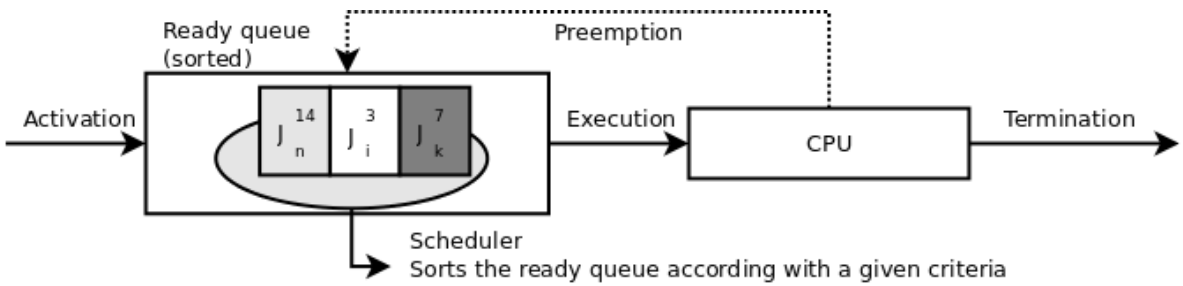
5 Other deadline assignment criteria

On-line scheduling with dynamic priorities

Dynamic priorities

Ready Queue sorted by instantaneous priorities and dynamically re-sorted (if/whenever necessary)

- Scheduling is based in dynamic criteria, i.e. one that is known only at run-time
- The dynamic parameter used to sort the ready tasks can be understood as a dynamic priority
- The ready queue is (re)sorted according with decreasing priorities whenever there is a priority change. Executes first the task that has the greater instantaneous priority



On-line scheduling with dynamic priorities

Pros

- Scales well (wrt Static Cyclic Scheduling (SCS))
  - Changes made to the task set are immediately seen by the scheduler
- Accommodates easily sporadic tasks (wrt SCS)
- Allows higher utilization levels than Fixed Priorities

Cons

- More complex implementation (wrt SCS and FP)
- Higher overhead (wrt SCS and FP)
  - Re-sorting of ready queue; depends on the algorithm
- "Unpredictability" on overloads (wrt FP)
  - It is not possible to know a priory the order by which tasks will fail deadlines in the event of an overload

On-line scheduling with dynamic priorities

More relevant priority allocation approaches:

- Inversely proportional to the time to the deadline
  - EDF – Earliest Deadline First
    - Optimal among all dynamic priority criteria
    - Complexity per task: O(n) if Ready Queue implemented as list or O(n.log(n)) if RQ implemented as heap
- Inversely proportional to the laxity or slack
  - LST / LLF
    - Optimal among all dynamic priority criteria
    - Computational overhead much higher than EDF, as it requires frequent re-evaluation and comparison of laxity for all active tasks, leading to frequent preemptions
    - Some modified versions address this issue.

1 Preliminaries

2 Online scheduling with dynamic priorities

3 Analysis: CPU utilization bound

4 Analysis: CPU Load Analysis

5 Other deadline assignment criteria

On-line scheduling with dynamic priorities

Schedulability tests

- Since the schedule is built online it is important to determine a priori if a given task set meets or not its temporal requirements
- There are several types of schedulability tests:
  - Based on CPU utilization
  - Based on CPU load (processor demand)
  - Based on response time

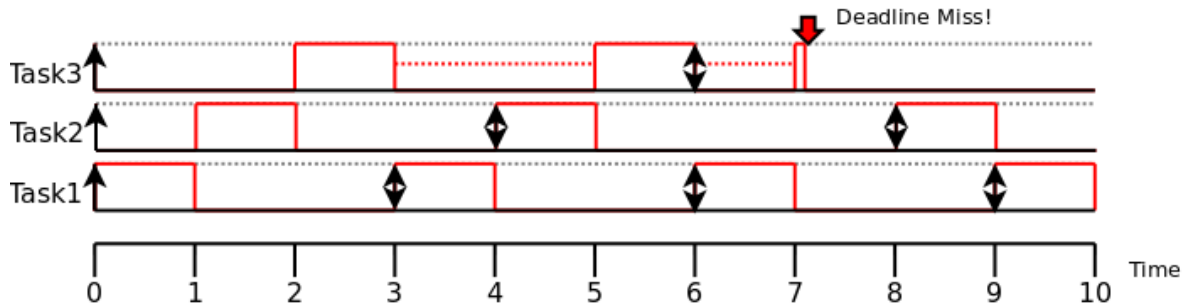
EDF Scheduling

EDF tests based on CPU utilization (n independent tasks, with full preemption)

- $D = T$
  - $U(n) = \sum_{i=1}^{n}(\frac{C_i}{T_i}) \leq 1 \Leftrightarrow$ Task set is schedulable
  - Allows using 100% of CPU with timeliness guarantees
- $D < T$
  - $U(n) = \sum_{i=1}^{n}(\frac{C_i}{D_i}) \leq 1 \Rightarrow$ Task set is schedulable
  - Sufficient condition, only.
  - Pessimistic test (as for RM, inflates the utilization)

RM Scheduling - example

τ	C	T
1	1	3
2	1	4
3	2.1	6

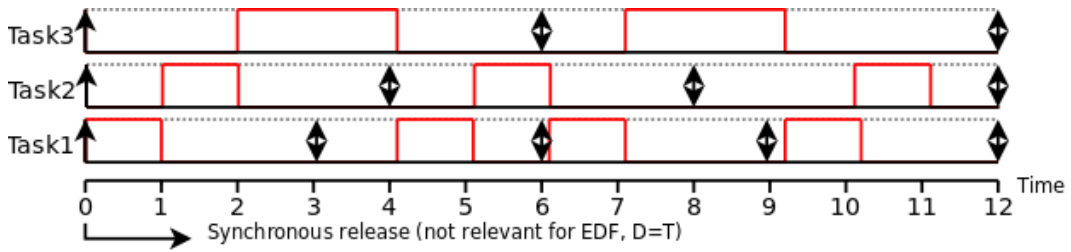


- $U = \frac{1}{3} + \frac{1}{4} + \frac{2.1}{6} = 0.93 > 0.78 \Rightarrow$ 1 activation per period NOT guaranteed.
- In fact τ_3 fails a deadline!

EDF Scheduling – same example

Same example as before!

τ	C	T
1	1	3
2	1	4
3	2.1	6

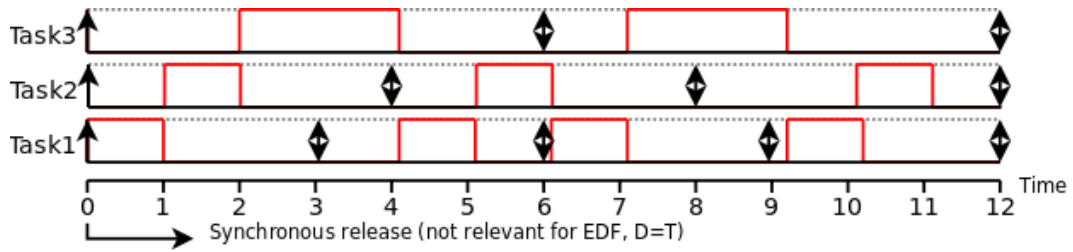


- $U = \frac{1}{3} + \frac{1}{4} + \frac{2.1}{6} = 0.93 \leq 1 \Leftrightarrow$ one activation per period IS guaranteed.

EDF Scheduling – same example

Same example as before!

τ	C	T
1	1	3
2	1	4
3	2.1	6



Note:

- No deadline misses
- Less preemptions
- Higher jitter on quicker tasks
- The worst-case response time does not coincide necessarily with the synchronous release

EDF Scheduling

Notion of **fairness**

- Be fair on the attribution of resources (e.g. CPU)
- EDF is intrinsically fairer than RM, in the sense that tasks see its relative deadline increased as the absolute deadline approaches, independently of its period or any other static parameter.
- Consequences:
  - Deadlines are easier to met
  - As the deadline approaches preemptions suffered by a given task are reduced
  - The slack of tasks that are quick but have large deadlines can be used by other task (higher jitter on tasks with shorter periods)

1 Preliminaries

2 Online scheduling with dynamic priorities

3 Analysis: CPU utilization bound

4 Analysis: CPU Load Analysis

5 Other deadline assignment criteria

CPU Load Analysis

- For $D < T$, the biggest period during which the CPU is permanently used (i.e. without interruption, idle time) corresponds to the scenario in which all tasks are activated synchronously. This period is called **synchronous busy period** and has duration L
- L can be computed by the following iterative method, which returns the first instant since the synchronous activation in which the CPU completes all the submitted jobs

Computation of L

CPU Load Analysis

- For $D < T$, the biggest period during which the CPU is permanently used (i.e. without interruption, idle time) corresponds to the scenario in which all tasks are activated synchronously. This period is called **synchronous busy period** and has duration L
- L can be computed by the following iterative method, which returns the first instant since the synchronous activation in which the CPU completes all the submitted jobs

Computation of L

$$L(0) = \sum_i (C_i)$$

$$L(m+1) = \sum_i (\lceil \frac{L(m)}{T_i} \rceil \cdot C_i)$$

Stop condition: $L(m+1) = L(m)$

CPU Load Analysis

- Knowing L, we have to guarantee the **Load Condition** , i.e.

$$\boxed{h(t) \leq t, \forall t \in [0, L[\Rightarrow \textit{ Task set is schedulable}}$$

- The Load Condition refers to all the work that must be **completed by t** .
- How to compute h(t)?

- Example:
  - Task set $\Gamma = \{(2,4),(2,8),(3,16)\}$
- Draw the Gantt chart and mark the points where tasks must complete
- Write a suitable equation

CPU Load Analysis

Knowing L we have to guarantee the load condition, i.e.

$$h(t) \leq t, \forall t \in [0, L[$$

And $h(t)$ can be computed as follows:

$$h(t) = \sum_{i=1..N} max(0, 1 + \lfloor \frac{t - D_i}{T_i} \rfloor) \cdot C_i$$

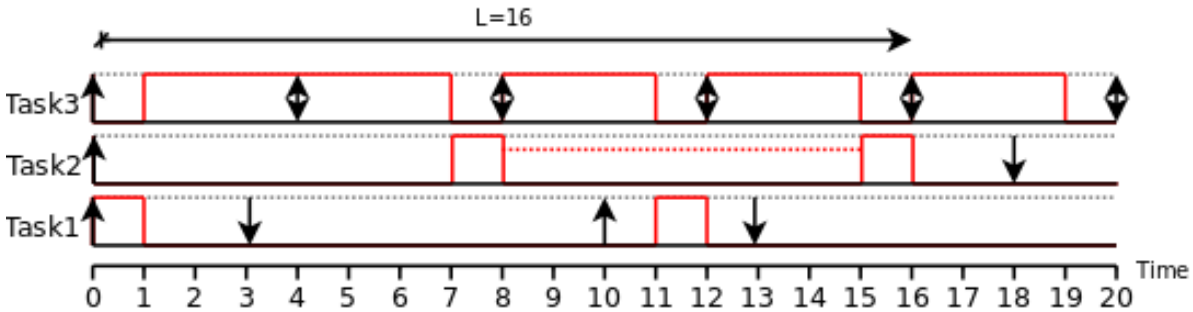
- The computation of h(t) for all values of $t \in [0, L[$ is unfeasible.
  - However it is enough computing the load condition for the instants in which the load function varies, i.e.

$$S = \bigcup_i (S_i), S_i = \{m \cdot T_i + D_i\}, m = 0, 1, ...$$

- Note: there are other, possibly shorter, values for L

EDF Scheduling

τ	C	D	T
1	1	3	10
2	2	18	20
3	3	4	4



- $\sum_{i=1..N} \frac{C_i}{D_i} = \frac{1}{3} + \frac{2}{18} + \frac{3}{4} = 1.194 > 1 \Rightarrow$ Schedulability not guaranteed by the utilization test
- But the CPU load analysis indicates that the task set is schedulable! Check it!

(Trying to) clarify the 1+floor() vs ceil() usage

Let's recap:

In CPU load analysis, the load function counts how much execution must be completed by a given time instant.

$$h(t) = \sum_{i=1}^{N} \max\left(0,\ 1 + \left\lfloor \frac{t - D_i}{T_i} \right\rfloor\right) C_i$$

Therefore, for each task, we must count how many jobs have absolute deadlines less than or equal to t.

For a task τ_i with relative deadline D_i and period T_i the absolute deadlines occur at:

$$D_i,\ D_i + T_i,\ D_i + 2T_i,\ D_i + 3T_i,\ \ldots$$

Why Not Use the Ceiling?

Why not use $\left\lceil \frac{t-D_i}{T_i} \right\rceil$ instead of $1 + \left\lfloor \frac{t-D_i}{T_i} \right\rfloor$?

- For non-integer values of x we have :

$$\lceil x \rceil = 1 + \lfloor x \rfloor$$

- So, between deadline instants, both expressions give the same value.
- **However, at exact deadline instants, they differ.**

Example

Consider a task with $D_i = 4$ and $T_i = 10$

- Its absolute deadlines are 4, 14, 24, ...
- At time $t = 4$ the first job must already be completed.

Using the correct expression, $1 + \left\lfloor \frac{t - D_i}{T_i} \right\rfloor$ we get:

- $1 + \left\lfloor \frac{4-4}{10} \right\rfloor = 1 + \lfloor 0 \rfloor = 1$
- So the first job is correctly counted.

Using the ceiling expression, $\left\lceil \frac{t - D_i}{T_i} \right\rceil$ we get:

- $\left\lceil \frac{4-4}{10} \right\rceil = \lceil 0 \rceil = 0$
- This is incorrect. The job whose deadline is exactly $t = 4$ is not counted.

1 Preliminaries

2 Online scheduling with dynamic priorities

3 Analysis: CPU utilization bound

4 Analysis: CPU Load Analysis

5 Other deadline assignment criteria

LLF/LST Scheduling

Least Laxity First : Executes first the task with smaller laxity

Laxity definition: $L_i(t)=d_i-t-c_i(t)$

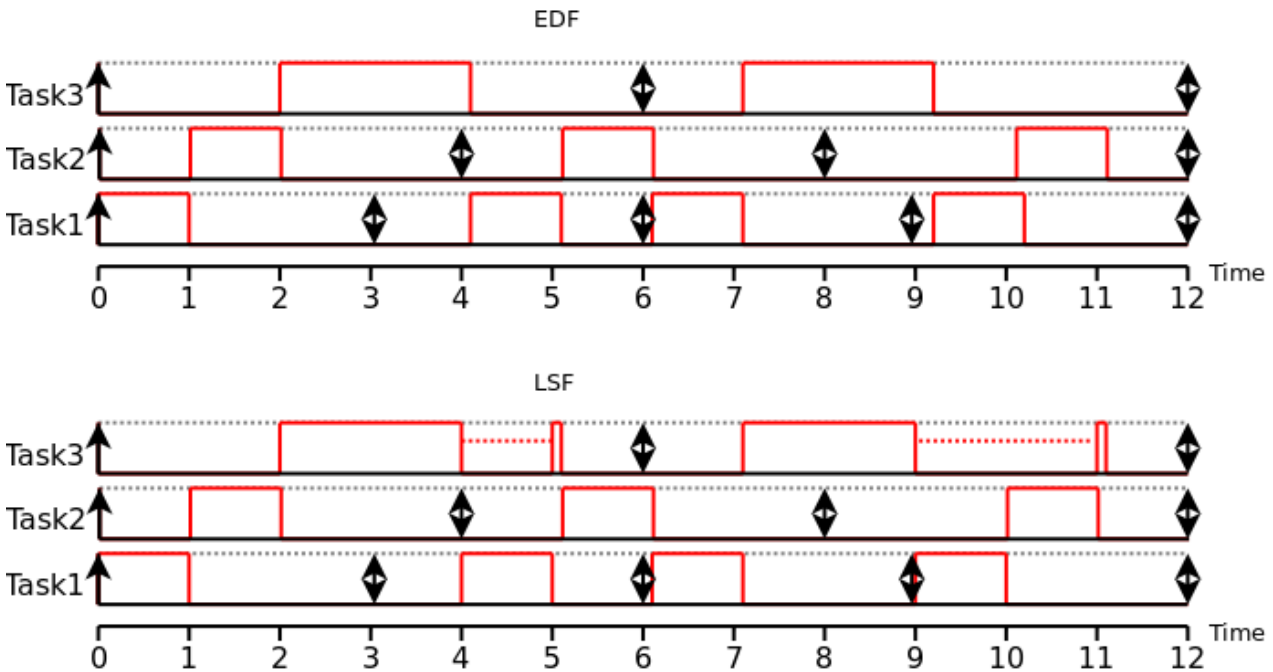
LLF vs EDF short comparison

- LLF is optimal (as EDF)
- As the laxity $\downarrow$ the priority $\uparrow$
- Priority of ready tasks increases as time goes by
  - Modifications to reduce the rescheduling points. **Why?**
- Priority of the task in the running state does not change
  - In EDF the priorities of all tasks (ready and executing) increase equally as time goes by
- Causes an higher number of preemptions than EDF (and thus higher overhead)
- No significant advantages with respect to EDF!

LLF scheduling example

LLF vs EDF

τ	C	T
1	1	3
2	1	4
3	2.1	6



Summary

- On-line scheduling with dynamic priorities
- The EDF - Earliest Deadline First criteria: CPU utilization bound and CPU Load Analysis
- Optimality of EDF and comparison with RM:
  - Schedulability level, number of preemptions, jitter and response time
- Other dynamic priority criteria:
  - LLF - Least Laxity First
  - Also optimal, but with higher overhead than EDF